

Python Development

Менеджер пакетов Pipenv (Pipenv)

Согласно статистике популярности библиотек Python для создания виртуальных окружений (virtual environments) на pypip-trends.com, самые популярные инструменты — **Pipenv** и **Poetry**. Оба упрощают работу с зависимостями (dependencies) в проектах на Python.

Pipenv — современный инструмент управления рабочим окружением (environment) в Python. Объединяет функционал `pip` и `virtualenv` в одном инструменте.

Основные возможности

- Создание и управление виртуальным окружением (virtual environment)
- Автоматическая синхронизация установленных/удалённых пакетов с файлом `Pipfile`
- Автоматическая подгрузка переменных окружения (environment variables) из файла `.env`

Какие проблемы решает

- **Хранение окружений в одном месте.** Все виртуальные окружения (virtual environments) хранятся в единой директории — удобно управлять, обновлять и удалять ненужные.
- **Управление зависимостями (dependency management).** Конфликты версий библиотек — частая проблема Python-разработчиков. Файл `requirements.txt` требует ручного контроля актуальности версий. Pipenv рекурсивно проверяет конфликты зависимостей и пытается установить последние совместимые версии.
- **Воспроизводимость окружения (environment reproducibility).** Для развёртывания (deployment) на другом компьютере нужно повторить точно такое же окружение. Pipenv фиксирует точные версии всех библиотек в отдельном файле — "слепок" рабочей системы.

Pipenv фактически стал стандартом среди Python-разработчиков — умение с ним работать обязательно.

Установка

```
pip install pipenv --user
```

Создание виртуального окружения

Pipenv определяет окружение по имени и полному пути директории проекта. Если нужно явно задать версию Python:

```
pipenv --python 3.12
```

Под капотом Pipenv использует `venv`, поэтому созданное окружение можно использовать и напрямую, без Pipenv.

Удаление окружения:

```
pipenv --rm
```

Установка пакетов

```
pipenv install Flask          # последняя версия
pipenv install Flask==0.6.1   # конкретная версия
```

Pipfile и Pipfile.lock

- `Pipfile` — какую версию пакета и откуда устанавливать.
- `Pipfile.lock` — все установленные пакеты (включая зависимости) вместе с их хешем (hash) и точной версией — гарантия, что версия пакета не изменилась ни на бит.

Из-за проверки хешей установка через `Pipenv` медленнее, чем через `pip`, зато надёжнее: `Pipenv` автоматически проверяет зависимости всех пакетов и старается поставить самые новые совместимые версии.

Зависимости для разработки (dev dependencies)

```
pipenv install ipython --dev
```

Пакет и все его зависимости помечаются как необязательные для финальной поставки (production).

Установка из существующих `Pipfile` / `Pipfile.lock`

Команда	Что делает
<code>pipenv install</code>	Устанавливает последние версии из <code>Pipfile</code> , может обновить <code>Pipfile.lock</code> , если вышли новые версии
<code>pipenv install --dev</code>	То же + пакеты, отмеченные как dev-зависимости
<code>pipenv sync</code>	Строго версии из <code>Pipfile.lock</code> , без проверки новых версий
<code>pipenv install --deploy</code>	Строго версии из <code>Pipfile.lock</code> ; ошибка, если хеши пакетов или версия Python не совпадают
<code>pipenv install --system</code>	Как обычный <code>install</code> , но пакеты ставятся глобально в систему, а не в виртуальное окружение

Удаление и обновление пакетов

```
pipenv uninstall package_name
```

`Pipenv` отслеживает зависимости и, в отличие от `pip`, удаляет пакет без "мусора" — неиспользуемых зависимостей, которые больше не нужны.

```
pipenv update # обновление всех пакетов до последних версий
```

Диагностика конфликтов зависимостей

Конфликт версий (version conflict) — когда один пакет требует версию библиотеки не выше X, а другой — не ниже Y, и это несовместимо. Стандартными средствами Python это не решить.

```
pipenv graph # список всех пакетов и их зависимостей
```

Если пакеты ставились в окружение напрямую через `pip`, лишнее (не указанное в `Pipfile.lock`) можно удалить командой:

```
pipenv clean
```

Менеджер пакетов Poetry (Poetry)

Poetry — инструмент для управления зависимостями (dependency management) в проектах на Python, аналог встроенного `pip`. По функциям похож на `Pipenv`, но более современный и гибкий: нацелен на упрощение всего цикла

управления проектом, включая публикацию пакета напрямую на **PyPI** (Python Package Index — репозиторий пакетов Python).

Зависимости проекта описываются в файле `pyproject.toml`, после чего Poetry устанавливает или обновляет их по запросу. Poetry также инкапсулирует проект в изолированное виртуальное окружение.

Связка файлов `pyproject.toml` и `poetry.lock` делает Poetry похожим на **npm** (Node Package Manager) в экосистеме Node.js.

Установка

Через `pip`:

```
pip install poetry
poetry --version # проверка установки
```

Через `pipx` (рекомендуется официальной документацией — изолирует инструменты на уровне системы и предотвращает конфликты версий):

```
pip install pipx
pipx ensurepath
pipx install poetry
```

Через официальный установщик (official installer):

```
# Windows (PowerShell)
(Invoke-WebRequest -Uri https://install.python-poetry.org -UseBasicParsing).Content | py -

# Linux, macOS, Windows (WSL)
curl -sSL https://install.python-poetry.org | python3 -
```

Poetry можно установить и вручную через `pip` и модуль `venv` (создать окружение → обновить `pip` / `setuptools` → `pip install poetry`) — это те же шаги, что выполняет официальный установщик. Для удаления достаточно удалить директорию окружения.

Создание нового проекта

```
poetry new solution
```

Результат — структура каталогов:

```
solution/
├─ solution/
│   └─ __init__.py
├─ tests/
│   └─ __init__.py
├─ pyproject.toml
└─ README.md
```

Любой Poetry-проект содержит минимум один пакет (package) — здесь это `solution`. Сразу создаётся и пакет `tests` с первым тестом: у реальных проектов всегда есть тесты.

Файл `pyproject.toml`

Это продвинутая альтернатива `requirements.txt`, формат файла — **TOML**.

```
[tool.poetry]
name = "solution"
version = "0.1.0"
description = ""
authors = ["FirstName LastName <youremail@gmail.com>"]

[tool.poetry.dependencies]
python = "^3.10"

[build-system]
requires = ["poetry-core"]
build-backend = "poetry.core.masonry.api"
```

- `[tool.poetry]` — метаданные проекта: имя, версия, описание, авторы.
- `[tool.poetry.dependencies]` — production-зависимости (нужны для работы приложения).
- `[tool.poetry.dev-dependencies]` (необязательный раздел) — зависимости для разработки, например `pytest`.

Инициализация в уже существующем проекте

```
poetry init
```

Запускает интерактивный опрос (имя пакета, версия, описание, автор, лицензия, зависимости) и генерирует `pyproject.toml`.

Активация виртуального окружения

```
poetry shell
```

Удобнее инициализировать проект через PyCharm — тогда терминал автоматически работает в виртуальном окружении. Нужно указать параметр **Poetry executable** (исполняемый файл Poetry), чтобы PyCharm знал, где он установлен.

Как найти путь к Poetry:

ОС	Команда
Windows (CMD)	<code>where poetry</code>
Windows (PowerShell)	<code>Get-Command poetry</code>
Linux	<code>which poetry</code>

Основные команды

```
poetry addaiosqlite
```

Добавляет зависимость и, если нужно, автоматически создаёт виртуальное окружение (хранятся в `~/.cache/pypoetry/virtualenvs`). Запись появляется в `pyproject.toml` → `[tool.poetry.dependencies]`.

```
poetry addpytest --dev
```

Добавление dev-зависимости — попадает в `[tool.poetry.dev-dependencies]`.

```
poetry removeaiosqlite  # удаление зависимости
poetry install           # установка зависимостей проекта
poetry update            # обновление всех зависимостей
poetry show              # список доступных пакетов
```

Если в проекте уже есть `poetry.lock`, команда `poetry install` ставит точные версии из него — это гарантирует одинаковые версии пакетов у всех участников проекта, независимо от того, что указано в `pyproject.toml`.

Версии зависимостей

Можно указать точную версию:

```
[tool.poetry.dependencies]
pygame = "2.1.0"
```

Либо диапазон версий с помощью специальных символов:

Синтаксис	Значение
"^2.1" (caret, "крышка")	Совместимые обновления — не меняет первую значащую цифру версии
"~2.1" (tilde, "тильда")	Только patch-обновления в пределах указанной минорной версии
"2.1.*" (wildcard, "маска")	Любой patch-релиз в пределах минорной версии 2.1
"*"	Любая версия

Форматирование кода (Code Formatting)

Соблюдение стандарта **PEP8** (Python Enhancement Proposal 8 — предложение №8 по улучшению Python, описывающее стиль кода) и других принятых в сообществе стандартов формально необязательно. Но рано или поздно в большинстве команд, разрабатывающих на Python, встаёт вопрос о едином стиле форматирования кода.

Black

Black — пакет, ставший фактически золотым стандартом форматирования кода в Python. Строго следует PEP8, но добавляет ряд собственных правил, улучшающих читаемость кода. Настройка "под себя" возможна, но в 99% случаев достаточно настроек по умолчанию.

```
pip install black

# форматирование одного файла
black my_python_script.py

# форматирование целого пакета
black my_package
```

Большинство IDE (Integrated Development Environment — интегрированная среда разработки) позволяют настроить автоматический запуск Black при сохранении файла. В PyCharm достаточно установить и включить Black в настройках; для VSCode используется плагин **Black Formatter**.

Статические анализаторы кода (Static Analyzers / Linters)

Инструменты, которые находят опечатки и "нелепые" ошибки, допущенные из-за невнимательности. К таким проблемам относятся:

- неправильное имя переменной или аргумента
- неверный тип аргумента при вызове функции или метода
- слишком глубокая вложенность блоков кода (циклы, условия)
- нарушение стандартов PEP
- пропущенные или лишние импорты (imports)
- использование "приватных" (private) атрибутов

Из-за динамической природы Python статические анализаторы (static analyzers) не идеальны и иногда ошибаются (ложные срабатывания), но всё равно существенно снижают число ошибок по невнимательности.

Инструмент	Особенности
Pylint	Очень строгий анализатор, часто даёт ложные срабатывания (false positives), делает самую глубокую проверку кода
Flake8	Аналог Pylint, но работает значительно быстрее и реже ошибается ложно, хотя пропускает часть ошибок
MyPy	Проверяет корректность типов аргументов функций и методов, хорошо работает с подсказками типов (typing); предупреждает о потенциально неверном типе аргумента (например, передача <code>None</code> вместо строки)
Radon	Анализирует цикломатическую сложность (cyclomatic complexity) кода — предупреждает, когда обилие вложенных циклов и ветвлений усложняет восприятие кода

Типизация в Python (Typing)

Языки программирования бывают статически типизированными (statically typed — проверка типов на этапе компиляции) или динамически типизированными (dynamically typed — проверка типов во время выполнения). Также языки бывают слабо или строго типизированными (weakly / strongly typed) — строгая типизация задаёт более жёсткие правила присвоения значений и вызова функций. **Python — динамический язык со строгой типизацией.**

Начиная с Python 3.5, подсказки типов (type hints) официально стали частью языка (PEP 484). С их помощью линтеры (linter) и средства проверки кода могут заранее, до запуска программы, проверять согласованность переменных и их типов.

Python не использует подсказки типов во время выполнения программы (runtime) — вся информация о типах удаляется в момент запуска. Type hints предназначены для разработчика и инструментов проверки типов (IDE, линтер), а не для интерпретатора.

Аннотации типов (Type Annotations)

```
def my_mul(data: list) -> float:
    result = 1
    for num in data:
        result = result * num
    return result
```

Это сообщает средству проверки типов, что функция `my_mul` принимает аргумент `data` типа `list` и возвращает `float`. Вызов с неверным типом аргумента (например, `my_mul(1)` вместо списка) вызовет предупреждение в IDE, например в PyCharm: `Expected type 'list', got 'int' instead`.

Для проверки типов из консоли (без IDE) чаще всего используют пакет **mypy**:

```
pip install mypy
mypy main.py
```

Псевдонимы типов (Type Aliases)

Для списков, словарей и кортежей (tuples) иногда нужно указать, объекты какого типа они содержат. Для этого используется модуль **typing**.

```

from typing import List

Data = List[float | int]

def my_mul(data: Data) -> float:
    result = 1
    for num in data:
        result = result * num
    return result

```

`Data` — псевдоним типа: список значений `int` или `float`.

```

from typing import Dict

dict_of_users: Dict[int, str] = {
    1: "Jane",
    2: "Jon"
}

```

Объединения типов (Union)

`Union` задаёт, что значение может быть одним из нескольких типов:

```

from typing import Union

Number = Union[float, int]

def add(x: Number, y: Number) -> Number:
    return x + y

```

Обобщённые типы (Generics)

Дженерики (generics) избавляют от дублирования кода, когда одна и та же функция должна работать с разными типами данных.

```

from typing import TypeVar

T = TypeVar("T", int, str, float)

def calculator(x: T, y: T) -> T:
    return x + y

print(calculator(3, 5))
print(calculator("Hello", "World"))
print(calculator(3.5, 1.4))

```

```

# вывод:
8
HelloWorld
4.9

```

Тип Any

`Any` — специальный тип, совместимый с любым другим типом при проверке. Использовать его стоит осознанно — он снижает пользу от статической проверки типов.

```
from typing import TypedDict, Any

T = TypedDict("T", int, str, float)

def calculator(x: Any, y: Any) -> T:
    return x + y
```

Отладка кода (Debugging)

Отладчик (debugger) — специализированный инструмент разработки, который подключается к выполняющемуся приложению и позволяет проверять код: выполнять его построчно, останавливать выполнение в любой момент, смотреть состояние памяти, значения переменных, доступность модулей.

Все популярные IDE используют для этого модуль **pdb** — пакет для полноценной отладки приложения, поддерживающий точки останова (breakpoints) и пошаговое выполнение.

```
# запуск из консоли
python3 -m pdb myscript.py

# запуск из интерпретатора Python
import pdb
import mymodule
pdb.run('mymodule.test()')
```

На практике удобнее пользоваться встроенным отладчиком своей IDE (PyCharm, VSCode), а не вызывать pdb напрямую.

Логирование приложения (Logging)

Даже после тестов и отладки ошибки случаются — особенно в продакшене (production) при работе с реальными пользователями, которые используют приложение не так, как ожидалось. Чтобы разобраться в причине, нужно записывать состояние приложения в журнал (log). Процесс называется логированием (logging).

Python предоставляет для этого модуль **logging** — гибкий и мощный инструмент, который можно настроить как в пару строк, так и в сложную многоуровневую систему.

Простейший пример — вывод в консоль:

```
import logging

logging.warning('This is a warning!')
```

```
# в консоли:
WARNING:root:This is a warning!
```

Более детальная настройка с записью в файл и в консоль одновременно:

```
import logging

logging.basicConfig(
    format='%(asctime)s %(message)s',
    level=logging.DEBUG,
    handlers=[
        logging.FileHandler("program.log"),
        logging.StreamHandler()
    ])
logging.warning('An example message.')
logging.warning('Another message')
```

Здесь заданы два обработчика (handlers): `StreamHandler` выводит в консоль, `FileHandler("program.log")` — записывает в файл `program.log`.

Чем logging лучше print

- показывает, в каком модуле и функции произошло событие
- пишет одновременно в несколько мест (файл, консоль, удалённый хост и т.д.)
- потокобезопасен (thread-safe) — сообщения из разных потоков не перемешиваются, в отличие от `print`
- уровень детализации логов меняется одним параметром на весь проект
- умеет вести журнал по кругу (rotating) — перезаписывая самые старые записи или после достижения определённого размера файла

Ключевые понятия logging

Понятие	Значение
уровень логирования (log level)	число от 0 до 50, показывающее важность записи
formatter	отвечает за формат сообщения: какие поля выводятся и как выглядит запись
handler	отвечает за обработку каждого сообщения; содержит formatter, уровень логирования и инструкцию, что делать с сообщением

Уровень	Числовое значение
DEBUG	10
INFO	20
WARNING	30
ERROR	40
CRITICAL	50

`handler` обрабатывает сообщение, только если его уровень равен или выше минимального уровня, заданного для этого handler'a. Так можно, например, писать в консоль вообще все сообщения, а в файл — только начиная с уровня ERROR:

```
import logging

# создаём логгер, задаём имя и уровень DEBUG
logger = logging.getLogger('simple_example')
logger.setLevel(logging.DEBUG)

# handler для консоли, уровень DEBUG
ch = logging.StreamHandler()
ch.setLevel(logging.DEBUG)

# formatter: время (asctime), имя модуля (name), уровень (levelname), сообщение (message)
formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')
ch.setFormatter(formatter)
logger.addHandler(ch)

# handler для файла, уровень ERROR
fh = logging.FileHandler("app.log")
fh.setLevel(logging.ERROR)
fh.setFormatter(formatter)
logger.addHandler(fh)

logger.debug('debug message')
logger.info('info message')
logger.warning('warn message')
logger.error('error message')
logger.critical('critical message')
```

В консоли появятся сообщения всех уровней, а в файле `app.log` — только `ERROR` и `CRITICAL`.

Паттерны проектирования (Design Patterns)

Паттерн проектирования — популярное решение типовой задачи при проектировании архитектуры программ. Это не конкретный код, а общая концепция или алгоритм, который реализуется под требования вашей программы. Если алгоритм — это точный кулинарный рецепт, то паттерн — это чертёж здания: общая схема, а конкретная реализация может отличаться от проекта к проекту.

По назначению паттерны делятся на три группы:

- **Порождающие (Creational)** — гибкое создание объектов без лишних зависимостей в программе
- **Структурные (Structural)** — способы построения связей между объектами
- **Поведенческие (Behavioral)** — эффективная коммуникация между объектами

Абстрактные базовые классы (Abstract Base Class, ABC)

Абстрактный класс предназначен для наследования, но не реализует методы полностью — оставляет только их сигнатуры, которые обязаны переопределить подклассы. Активно используется при написании паттернов.

Простейший вариант:

```
class MyABC:
    def foo(self):
        raise NotImplementedError()

    def bar(self):
        raise NotImplementedError()
```

Минус такого подхода: исключение возникнет, только если реально вызвать неопределённый метод — легко забыть переопределить один из методов, и ошибка обнаружится не сразу.

Надёжнее использовать модуль **abc**: метакласс `ABCMeta` для создания абстрактного класса и декоратор `@abstractmethod` для обязательных методов. Тогда ошибка (`TypeError`) возникнет уже в момент создания объекта наследника, если он не реализовал все абстрактные методы:

```

from abc import abstractmethod, ABCMeta

class MyBaseClass(metaclass=ABCMeta):
    @abstractmethod
    def foo(self):
        pass

    @abstractmethod
    def baz(self):
        pass

class Child(MyBaseClass):
    pass

c = Child()
# TypeError: Can't instantiate abstract class Child with abstract methods baz, foo

```

Метакласс (metaclass) — класс, экземплярами которого являются другие классы. Свой метакласс создаётся как подкласс `type` — стандартного метакласса Python: `ExampleClass = type('ExampleClass', (object,), {'some_var': 1})`. Метаклассы нужны для метапрограммирования (динамическая генерация классов во время выполнения) и в 99.9% случаев не требуются — почти всегда есть более простой способ добиться того же.

Проще и без метаклассов того же эффекта позволяет достичь наследование от `ABC`: `class MyBaseClass(ABC):` вместо `metaclass=ABCMeta` — результат идентичен.

Python — язык с динамической типизацией, поэтому объект можно использовать независимо от его класса, если у него есть нужные атрибуты. Это удобно на старте проекта, но с ростом кодовой базы повышает риск ошибок. Абстрактные базовые классы делают код само-документируемым и снижают вероятность забыть или неверно реализовать нужный метод. Типичный пример — разные форматы генерации документов (pdf, html, txt):

```

from abc import ABC, abstractmethod

class Document(ABC):
    @abstractmethod
    def generate(self, data):
        pass

```

Любой класс конкретного формата теперь обязан наследовать от `Document` и реализовать `generate` — добавлять новые форматы станет проще и безопаснее.

Порождающие паттерны (Creational Patterns)

Абстрактная фабрика (Abstract Factory)

Позволяет создавать семейства связанных объектов, не привязываясь к конкретным классам. Пример: нужно генерировать отчёты (месячный, квартальный, годовой) в разных форматах (pdf, html, csv).

```

from abc import ABC, abstractmethod

class AbstractReport(ABC):
    @abstractmethod
    def create_month_report(self):
        pass

    @abstractmethod
    def create_quarter_report(self):
        pass

    @abstractmethod
    def create_year_report(self):
        pass

class PdfReport(AbstractReport):
    def create_month_report(self):
        return PdfMonthReport()

    def create_quarter_report(self):
        return PdfQuarterReport()

    def create_year_report(self):
        return PdfYearReport()

class HtmlReport(AbstractReport):
    def create_month_report(self):
        return HtmlMonthReport()
    # ... аналогично create_quarter_report, create_year_report

```

Абстрактная фабрика `AbstractReport` задаёт, какие периоды отчётов обязаны уметь генерировать конкретные фабрики, а сами фабрики возвращают классы отчётов нужного формата и периода. Понадобится новый период — измените только `AbstractReport`, а IDE подскажет, какие классы-наследники ещё не реализовали новый метод.

Не стоит объединять маленькие классы вроде `CsvMonthReport`, `CsvQuarterReport` в один большой универсальный класс — компактные узкоспециализированные классы предпочтительнее.

Фабричный метод (Factory Method)

Определяет общий интерфейс создания объектов в суперклассе, позволяя подклассам менять тип создаваемых объектов. Вместо создания объектов напрямую, они создаются через специальный фабричный метод — все создаваемые объекты имеют общий интерфейс.

```

from abc import ABC, abstractmethod

class Creator(ABC):
    @abstractmethod
    def create(self):
        pass

    def send_messages(self) -> str:
        product = self.create()
        result = product.sending()
        return result

class SendingMessages(ABC):
    @abstractmethod
    def sending(self) -> str:
        pass

class CreatorPush(Creator):
    def create(self) -> SendingMessages:
        return SendingPushMessages()

class CreatorSMS(Creator):
    def create(self) -> SendingMessages:
        return SendingSMSMessages()

class SendingPushMessages(SendingMessages):
    def sending(self) -> str:
        return "Push mailing has been completed"

class SendingSMSMessages(SendingMessages):
    def sending(self) -> str:
        return "SMS mailing has been completed"

def client_code(creator: Creator) -> None:
    result = creator.send_messages()
    print(f"Result: {result}")

client_code(CreatorPush())    # Result: Push mailing has been completed
client_code(CreatorSMS())    # Result: SMS mailing has been completed

```

Так добавление нового канала рассылки (email, telegram) не требует переписывать существующий код — достаточно добавить новую пару классов `Creator` / `SendingMessages`.

Одиночка (Singleton)

Гарантирует, что у класса есть только один экземпляр, и предоставляет к нему глобальную точку доступа. Решает две задачи: единственный экземпляр класса и глобальный доступ к нему.

```

import random

class Singleton:
    __instance = None

    def __init__(self):
        self.number = random.randint(1, 10)

    def __new__(cls):
        if cls.__instance is None:
            cls.__instance = super().__new__(Singleton)
        return cls.__instance

list_singleton = [Singleton() for i in range(5)]
# у всех пяти элементов будет одно и то же случайное число,
# т.к. это фактически один и тот же объект

```

Обычный класс на тех же пяти созданиях вернул бы пять разных случайных чисел. Singleton чаще всего применяется для доступа к общему ресурсу, например к базе данных.

Структурные паттерны (Structural Patterns)

Фасад (Facade)

Предоставляет простой интерфейс к сложной системе классов, библиотеке или фреймворку. Решает проблему, когда бизнес-логика тесно переплетается с деталями реализации сторонних библиотек, отчего код становится запутанным. Сложная логика прячется за простым интерфейсом — как сайт интернет-магазина скрывает от покупателя склад, накладные и логистику.

```
class FacadeNewsletter:
    def __init__(self, users_system, email_system) -> None:
        self._users_system = users_system
        self._email_system = email_system

    def sending(self) -> str:
        users = self._users_system.get_users()
        male, female = self._users_system.separate_users(users)
        text_for_male = self._email_system.get_text_email("male")
        text_for_female = self._email_system.get_text_email("female")
        self._email_system.send_emails(male, text_for_male)
        self._email_system.send_emails(female, text_for_female)
        return "Done"

class UsersSystem:
    def get_users(self) -> list:
        return [
            {"name": "Allen Raymond", "gender": "male"},
            {"name": "Kennedy Lane", "gender": "female"},
        ]

    def separate_users(self, users) -> tuple:
        male = [p for p in users if p["gender"] == "male"]
        female = [p for p in users if p["gender"] != "male"]
        return male, female

class EmailSystem:
    def get_text_email(self, gender) -> str:
        return "Male text email" if gender == "male" else "Female text email"

    def send_emails(self, users, text) -> str:
        for person in users:
            print(f"Send {person['name']} email: {text}")
        return "Done"

facade = FacadeNewsletter(UsersSystem(), EmailSystem())
facade.sending()
```

Клиентский код работает только с `FacadeNewsletter` и не знает, что внутри задействованы две отдельные системы — пользователей и рассылки.

Адаптер (Adapter)

Позволяет объектам с несовместимыми интерфейсами работать вместе — выступает переходником, превращая интерфейс одного класса в интерфейс, ожидаемый другим классом.

```

class OldPrinter:
    def print_text(self, text):
        print(f"OldPrinter: {text}")

class NewPrinter:
    def print(self, text):
        print(f"NewPrinter: {text}")

class PrinterAdapter:
    def __init__(self, new_printer: NewPrinter):
        self.new_printer = new_printer

    def print_text(self, text):
        self.new_printer.print(text)

def client_code(printer: OldPrinter):
    printer.print_text("Hello, World!")

adapter = PrinterAdapter(NewPrinter())
client_code(adapter)  # NewPrinter: Hello, World!

```

`OldPrinter` ожидает метод `print_text`, а у `NewPrinter` есть только `print`. Адаптер `PrinterAdapter` оборачивает `NewPrinter` и предоставляет клиентскому коду ожидаемый метод `print_text` — полезно, чтобы использовать новые классы в старых системах без изменения существующего кода.

Заместитель (Proxy)

Позволяет подставить вместо реального объекта объект-заменитель с тем же интерфейсом. Частые применения: ленивая загрузка (lazy loading), кэширование (caching), контроль доступа, логирование.

```

from abc import ABC, abstractmethod
from time import time, sleep

class Request(ABC):
    @abstractmethod
    def request(self) -> None:
        pass

class RealRequest(Request):
    def request(self) -> None:
        print("RealRequest: Handling request.")
        sleep(0.5)

class Proxy(Request):
    def __init__(self, real_request) -> None:
        self._real_request = real_request
        self.start = None

    def request(self) -> None:
        self.start = time()
        self._real_request.request()
        self.log_access()

    def log_access(self) -> None:
        print(f"Proxy: Logging the time of request. {time() - self.start}")

proxy = Proxy(RealRequest())
proxy.request()

```

Поведенческие паттерны (Behavioral Patterns)

Решают задачи эффективного и безопасного взаимодействия между объектами программы.

Команда (Command)

Превращает запросы в объекты, позволяя передавать их как аргументы, ставить в очередь, логировать и поддерживать отмену операций.

```
from abc import ABC, abstractmethod

class Command(ABC):
    @abstractmethod
    def execute(self) -> None:
        pass

class CommandCreateXMLOrder(Command):
    def __init__(self, receiver, text: str) -> None:
        self._receiver = receiver
        self._text = text

    def execute(self) -> None:
        self._receiver.createXMLOrder(self._text)

class CommandSendEmail(Command):
    def __init__(self, receiver, html: str) -> None:
        self._receiver = receiver
        self._html = html

    def execute(self) -> None:
        self._receiver.send_email(self._html)

class Receiver:
    def createXMLOrder(self, text: str) -> None:
        print(f"Create XML order: {text}")

    def send_email(self, text: str) -> None:
        print(f"Send email: {text}")

class Invoker:
    def __init__(self) -> None:
        self._on_order = None
        self._on_email = None

    def set_on_order(self, command: Command):
        self._on_order = command

    def set_on_email(self, command: Command):
        self._on_email = command

    def generate_general_order(self) -> None:
        self._on_order.execute()
        self._on_email.execute()

invoker = Invoker()
invoker.set_on_order(CommandSendEmail(Receiver(), "Send email"))
invoker.set_on_email(CommandCreateXMLOrder(Receiver(), "Save report"))
invoker.generate_general_order()
```

Роли: **Command** — общий интерфейс команд (обычно один метод запуска); конкретные команды (`CommandCreateXMLOrder` , `CommandSendEmail`) реализуют разные запросы и обычно делегируют выполнение **Receiver** (получателю) — объекту с бизнес-логикой; **Invoker** (отправитель) хранит ссылки на команды и вызывает их через общий интерфейс, не зная деталей конкретной команды; **клиент** создаёт команды и настраивает ими отправителя.

Наблюдатель (Observer)

Создаёт механизм подписки: одни объекты следят и реагируют на события в других объектах. Аналогия: кнопка "сообщить о поступлении товара" в интернет-магазине — вместо постоянной проверки сайта покупатель подписывается и получает уведомление только когда товар появился. Магазин (издатель, publisher) хранит список подписчиков (observers) и оповещает их при изменении своего состояния.

```

from abc import ABC, abstractmethod

class Publisher(ABC):
    @abstractmethod
    def attach(self, observer): pass
    @abstractmethod
    def detach(self, observer): pass
    @abstractmethod
    def notify(self): pass

class PublisherMessages(Publisher):
    _observers = []
    _indicator = 0

    def attach(self, observer):
        self._observers.append(observer)

    def detach(self, observer):
        self._observers.remove(observer)

    def notify(self):
        for observer in self._observers:
            observer.update(self)

    def business_logic_execution(self):
        self._indicator += 1
        self.notify()

class Observer(ABC):
    @abstractmethod
    def update(self, publisher): pass

class ObserverA(Observer):
    def update(self, publisher):
        if publisher._indicator <= 3:
            print("ObserverA: reacts to indicator <= 3")

class ObserverB(Observer):
    def update(self, publisher):
        if publisher._indicator > 2:
            print("ObserverB: reacts to indicator > 2")

publisher = PublisherMessages()
publisher.attach(ObserverA())
publisher.attach(ObserverB())
publisher.business_logic_execution() # indicator 1: реагирует ObserverA
publisher.business_logic_execution() # indicator 2: реагирует ObserverA
publisher.business_logic_execution() # indicator 3: реагируют оба

```

При каждом вызове `business_logic_execution` выполняется `notify()`, который проходит по списку подписчиков и вызывает у каждого `update`, передавая ссылку на издателя — так подписчик сам решает, реагировать на текущее состояние или нет.

Паттерны проектирования дают проверенные решения типовых задач, ускоряют разработку гибких и масштабируемых систем, формируют общий язык для обсуждения архитектуры между разработчиками и снижают количество ошибок в коде. Подробнее — на refactoring.guru.

Принципы проектирования SOLID

SOLID — аббревиатура из первых букв пяти базовых принципов объектно-ориентированного программирования (Object-Oriented Programming, ООП), предложенных Робертом Мартином (Robert Martin). Принципы помогают проектировать системы, которые долго остаются расширяемыми, масштабируемыми и поддерживаемыми.

Буква	Оригинал	Русский перевод
S	Single responsibility	Принцип единственной ответственности
O	Open-closed	Принцип открытости/закрытости
L	Liskov substitution	Принцип подстановки Барбары Лисков
I	Interface segregation	Принцип разделения интерфейса
D	Dependency inversion	Принцип инверсии зависимостей

S — принцип единственной ответственности (Single Responsibility Principle, SRP)

У модуля должна быть только одна причина для изменения — то есть одна бизнес-причина (например, требование конкретного отдела или группы пользователей). Модуль — любая обособленная единица кода: класс, функция, объект, файл.

Приём "выделение класса" (extract class): из большого класса со слабо связанными полями и методами выделяют один или несколько классов с чётким назначением. Признак правильно выделенного класса — понятное имя-существительное с прилагательным (если напрашивается отлагательное имя — назначение класса выбрано неверно).

```
# до: класс хранит данные и форматирует адрес — две ответственности
class Person:
    def __init__(self, name, zip, city, street):
        self.name, self.zip, self.city, self.street = name, zip, city, street

    def get_address(self):
        return f'{self.zip}, {self.city}, {self.street}'
```

```
# после: форматирование адреса вынесено в отдельный класс
class PersonAddress:
    def __init__(self, zip, city, street):
        self.zip, self.city, self.street = zip, city, street

    def value_of(self):
        return f'{self.zip}, {self.city}, {self.street}'

class Person:
    def __init__(self, name, address):
        self.name = name
        self.address = address

    def get_address(self):
        return self.address.value_of()

person = Person('Olexander', PersonAddress('36007', 'Poltava', 'European, 28'))
```

O — принцип открытости/закрытости (Open-Closed Principle, OCP)

Модули должны быть открыты для расширения, но закрыты для изменения. Новая функциональность добавляется новыми сущностями и их композицией со старыми, а не правкой уже работающего кода.

нарушение: при каждой новой фигуре нужно дописывать total_area

```
from math import pi

class Rect:
    def __init__(self, width, height):
        self.width, self.height = width, height

class Circle:
    def __init__(self, radius):
        self.radius = radius

def total_area(shapes):
    total = 0
    for el in shapes:
        if isinstance(el, Rect):
            total += el.width * el.height
        if isinstance(el, Circle):
            total += el.radius ** 2 * pi
    return total
```

исправление: каждая фигура сама умеет считать свою площадь

```
class Rect:
    def __init__(self, width, height):
        self.width, self.height = width, height

    def area_of(self):
        return self.width * self.height

class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area_of(self):
        return self.radius ** 2 * pi

def total_area(shapes):
    return sum(el.area_of() for el in shapes)
```

Теперь при появлении новой фигуры `total_area` менять не нужно — достаточно добавить класс с методом `area_of`.

L — принцип подстановки Барбары Лисков (Liskov Substitution Principle, LSP)

Объекты подклассов должны быть взаимозаменяемы с объектами базового класса без нарушения корректности программы: если `B` — подкласс `A`, экземпляры `A` можно свободно заменять экземплярами `B`.

Классический пример нарушения — прямоугольник и квадрат:

```

class Rectangle:
    def __init__(self, width, height):
        self._width, self._height = width, height

    def set_width(self, width): self._width = width
    def set_height(self, height): self._height = height
    def area(self): return self._width * self._height

class Square(Rectangle):
    def __init__(self, side):
        super().__init__(side, side)

    def set_width(self, width):
        self._width = self._height = width    # меняет оба измерения!

    def set_height(self, height):
        self._width = self._height = height

def resize_rectangle(rect: Rectangle):
    rect.set_width(10)
    rect.set_height(20)
    assert rect.area() == 200

resize_rectangle(Rectangle(2, 3))    # работает
resize_rectangle(Square(5))          # AssertionError: площадь 400, а не 200

```

`Square` переопределяет `set_width` / `set_height` так, что нарушает контракт базового класса `Rectangle`. Решение — не наследовать квадрат от прямоугольника, а реализовать оба через общий интерфейс:

```

from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self): pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width, self.height = width, height
    def area(self): return self.width * self.height

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self): return self.side * self.side

```

Теперь `Rectangle` и `Square` не наследуют друг друга, а реализуют общий интерфейс `Shape` — принцип подстановки не нарушается.

Второй классический пример — птицы:

```

# нарушение: не все птицы умеют летать
class Bird:
    def fly(self): pass

class Sparrow(Bird):
    def fly(self): print("Sparrow is flying")

class Penguin(Bird):
    def fly(self): raise Exception("Penguins cannot fly")    # нарушает контракт Bird

```

```
# исправление: отдельный интерфейс для летающих птиц
from abc import ABC, abstractmethod

class Bird(ABC):
    @abstractmethod
    def move(self): pass

class FlyableBird(Bird):
    def move(self): self.fly()
    @abstractmethod
    def fly(self): pass

class Sparrow(FlyableBird):
    def fly(self): print("Sparrow is flying")

class Penguin(Bird):
    def move(self): print("Penguin is walking")
```

Теперь общий контракт `Bird` требует только `move()` — способ передвижения у каждого подкласса свой, и подстановка любого подкласса вместо `Bird` больше не ломает программу.

I — принцип разделения интерфейса (Interface Segregation Principle, ISP)

Сущности не должны зависеть от интерфейсов, которые они не используют. Нарушение принципа делает модули зависимыми от изменений в чужой, не нужной им части интерфейса — это повышает связанность (coupling) кода. Решение — дробить широкие интерфейсы на более узкие.

```
# нарушение: RemoteProgrammer вынужден реализовать ненужный eat_pizza
class Programmer:
    def write_code(self): pass
    def eat_pizza(self, slice_count): pass
```

```
# исправление: роли разделены на два узких интерфейса
class CodeProducer:
    def write_code(self): pass

class PizzaConsumer:
    def eat_pizza(self, slice_count): pass

class OfficeProgrammer(CodeProducer, PizzaConsumer):
    def __init__(self, name): self.name = name
    def write_code(self): print(f'{self.name} write code!')
    def eat_pizza(self, slice_count): print(f'{self.name} eat {slice_count} slice pizza!')

class RemoteProgrammer(CodeProducer):
    def __init__(self, name): self.name = name
    def write_code(self): print(f'{self.name} write code!')
```

`RemoteProgrammer` теперь реализует только тот интерфейс (`CodeProducer`), который ему реально нужен.

D — принцип инверсии зависимостей (Dependency Inversion Principle, DIP)

Классы должны зависеть не друг от друга напрямую, а от абстракций. Высокоуровневые классы не должны зависеть от низкоуровневых — оба должны зависеть от абстракций; абстракции не должны зависеть от деталей реализации, а детали — зависеть от абстракций.

```

import requests

class RequestConnection:
    # абстракция поверх конкретной HTTP-библиотеки
    def __init__(self, request):
        self.request = request

    def get_json_from_url(self, url):
        return self.request.get(url).json()

class ApiClient:
    # высокоуровневый класс зависит от абстракции, а не от requests напрямую
    def __init__(self, fetch: RequestConnection):
        self.fetch = fetch

    def get_data(self, url):
        return self.fetch.get_json_from_url(url)

api_client = ApiClient(RequestConnection(requests))
data = api_client.get_data('https://api.privatbank.ua/p24api/pubinfo?exchange&coursid=11')

```

`ApiClient` зависит от абстракции `RequestConnection`, а не от пакета `requests` напрямую. Благодаря этому библиотеку HTTP-запросов внутри `RequestConnection` можно заменить на другую, не трогая код `ApiClient`.